

RDHIS2 Mobile

Technical Documentation

Offline-first Flutter application for aggregate health-data capture into DHIS2

HISP Ethiopia · in collaboration with the Federal Ministry of Health

Document version 1.0 · 10 September 2026

Audience: engineering, technical reviewers, HISP / Ministry stakeholders

Document Control

Field	Value
Title	RDHIS2 Mobile — Technical Documentation
Version	1.0
Date	10 September 2026
Status	Baseline — reflects the codebase at the current development head
Owner	HISP Ethiopia — Mobile team
Application	RDHIS2 Mobile App (package: hisp_mobile_trucker), app version 1.0.0+1
Source of truth	The repository. Every claim here is traceable to a file path; where behaviour and this document disagree, the code is authoritative.
Companion docs	docs/ (16+ developer-onboarding chapters); OFFLINE_INTEGRATION.md (historical design note); ministry_presentation/ (stakeholder deck)

Scope.

This document describes what the application is, how it is built, how the offline-first model works end to end, the data-quality and synchronization machinery, the security posture, the database schema, and the build / release / test process. It is written to be read by a newcomer to the project and by a technical reviewer who will not contribute code but needs an accurate picture of the system.

Change history.

Version	Date	Notes
1.0	2026-09-10	First consolidated technical documentation. Supersedes the point-in-time notes in ARCHITECTURE_ANALYSIS.md and folds in features added since the docs/ set was written (data-quality suite, outlier detection, audit trail, schema migrations, local dashboards, deadline reminders, onboarding, background-sync hardening).

1 Executive Summary

RDHIS2 Mobile is an Android / iOS / web application, built in Flutter, that lets health workers record aggregate service-delivery data and submit it to DHIS2 — the national Health Management Information System. It is designed for the conditions the data is actually collected in: facilities with no connectivity, intermittent connectivity, or a connection too weak to depend on.

The one-sentence mental model

Every screen reads from and writes to a **local, per-user SQLite database first**. The network is not a dependency for daily work — it is how that local database is seeded (metadata sync) and how locally-queued entries eventually reach the DHIS2 server (data sync). **The local database is the truth for the UI; the server is a sync target, not a live dependency.**

What it does

- Online and offline login. The first login is verified against the DHIS2 server; afterwards the same credentials work offline indefinitely, checked locally against a salted hash that never stores the password.
- Offline-first capture. Organisation-unit tree → dataset → section → period → entry form. Every keystroke is saved locally. Nothing is lost when the network drops mid-form.
- Draft → Complete workflow. Entries stay as device-only drafts until the user marks the form Complete; only then are they queued for the server. Completed forms can be reopened for correction.
- Automatic and manual synchronization, with per-value server verdicts. Values the server rejects are shown in red in the form with the server's own reason; editing them re-queues the fix.
- Data quality at the point of entry: client-side value-type validation, offline evaluation of the server's validation rules, mandatory-field enforcement, grey (disabled) fields, controller-element gating, and a statistical outlier check against the cell's own recent history.
- A per-cell audit trail combining this device's local edit history with DHIS2's own server-side audit log.
- Native dashboards. The Visualization area renders DHIS2 server dashboards with Flutter charts, lets the user build and save local dashboards on the device, and caches results so a chart is still viewable offline.
- Ethiopian-calendar period handling throughout, including the DHIS2 Ethiopian-fiscal-year period types.
- On-device deadline reminders for reports the user still owes, and a first-run onboarding tour.

Project size (current head)

Metric	Value
Production Dart	~29,000 lines across ~160 files (excludes generated code)
Generated code	~16,900 lines (Drift database, committed to the repo)
Automated tests	24 test files, ~5,200 lines — weighted toward the offline-critical core (sync, conflict resolution, validation, calendar)
Platforms	Android (primary), iOS, Web (development / demo)

Toolchain	Flutter (CI pins 3.41.4), Dart SDK $\geq 3.0.0 < 4.0.0$

Security posture in one paragraph

Passwords are never stored in any recoverable form, online or offline — offline login is checked against a salted SHA-256 verifier bound to the server URL, compared in constant time. Session and credential material live in OS-backed secure storage (Android Keystore-backed EncryptedSharedPreferences / iOS Keychain), never in the SQLite database. Transport is HTTPS-only in release builds. The known hardening gaps — local database not encrypted at rest (it holds cached metadata and pending field data, never credentials), no certificate pinning, no R8/ProGuard — are defense-in-depth items scoped to the “lost or stolen device” threat model, each with a concrete fix on the roadmap. Full detail in section 9.

2 System Overview

2.1 Purpose and context

DHIS2 is the platform of record for routine health data in Ethiopia. Its web and official Android clients assume a usable connection at the moment of data entry. In much of the country that assumption does not hold: a health worker may capture a full day or month of data with no connectivity and only reach a network hours or days later. RDHIS2 Mobile exists to make that workflow safe — capture now, sync later, lose nothing, and never silently overwrite work.

2.2 Users and roles

- Facility data-entry users — the primary audience. Assigned a small organisation-unit scope (their facility, sometimes a woreda and its direct children) and a set of datasets. They capture, complete, and sync reports.
- Supervisory / woreda users — a wider org-unit scope; the same capture flow plus dashboards spanning their subtree.

There is no in-app role management: the app inherits whatever the DHIS2 server grants the authenticated user (org-unit capture scope, dataset access, metadata visibility). The app never widens that scope.

2.3 DHIS2 integration surface

The app speaks the standard DHIS2 Web API over HTTP Basic authentication. The endpoints it uses, and nothing else:

Purpose	Endpoint(s)	Direction
Credential check on login	GET /api/me.json	read
Metadata sync	GET /api/{resource}.json (categories, category combos/options, option sets, attributes, data elements, indicators, data element groups, organisation units, data sets, sections, validation rules)	read
Form open — pull current values	GET /api/dataValueSets.json	read
Outlier history snapshot	GET /api/dataValueSets.json (date- windowed)	read
Data push	POST /api/dataValueSets.json (importStrategy CREATE_AND_UPDATE, atomicMode NONE)	write
Completion push	POST / DELETE /api/completeDataSetRegistrations	write
Server audit trail	GET /api/audits/dataValue.json	read
Connectivity probe	GET /api/system/ping (or	read

	{baseUrl}/system/ping)	
Dashboards & analytics	GET /api/dashboards, /api/visualizations/{id}, /api/analytics, /api/indicatorGroups, /api/dataElementGroups	read

The app never creates or edits a server-side Visualization, dashboard, metadata object, or user. Its only writes are data values and completion registrations.

2.4 Server configuration

The compiled-in default server is `https://hmis-staging.moh.gov.et/api` (`lib/core/constants/api_constants.dart`). Users and developers can repoint the app at any DHIS2 instance at runtime — no rebuild — from the server icon on the login screen or Settings → DHIS2 server URL. The override is normalised to `https:// + /api` and persisted in secure storage.

3 Architecture

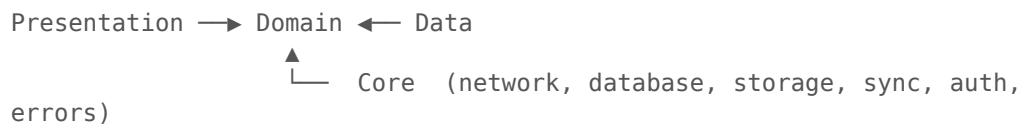
3.1 Style: feature-first Clean Architecture

Each feature under `lib/features/` owns its own data / domain / presentation split. Cross-cutting infrastructure lives in `lib/core/`; shared UI in `lib/shared/`. The three-layer split is applied where there is a real repository / use-case boundary to draw, not as ceremony: home and settings are presentation-only because they compose other features and own no persistence.

The four layers, and why each exists

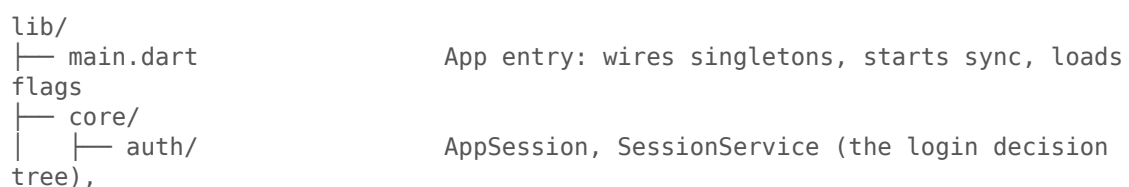
Layer	Contents	Rationale
Presentation	Pages, widgets, Bloc classes. Owns UI state, translates user intent into domain calls. Never touches Dio or Drift directly.	A screen can be redesigned or rebuilt without touching business rules.
Domain	Entities, use cases, repository interfaces. Pure Dart — no Flutter, Dio, or Drift imports anywhere in a domain folder.	Business rules are testable in total isolation and would survive a rewrite of the UI, the network stack, or the database.
Data	Repository implementations, remote/local data sources, models (DTOs with <code>fromJson/toJson</code> that extend the domain entities).	One place per feature holds its persistence strategy. Auth and data-entry each migrated from remote-only to offline-first here without a single Bloc or page changing.
Core	HTTP client, database, secure storage, sync engine, session/auth, shared exception types, the Ethiopian calendar, data-quality services.	One source of truth for “is the user logged in”, “are we online”, “what period is it” — no feature reinvents them.

Dependency direction



Presentation and Data both depend on Domain; Domain depends on nothing feature-specific. A Bloc holds a repository interface, never the concrete implementation — which is what makes it possible to inject a fake in a test.

3.2 Module map



- constants/ - data/	CredentialStore (offline verifier) API + app constants DataValueStore/Sync/Push, Completeness,
AuditLogStore,	
ControllerElementService,	ServerAuditService, ValidationService, OutlierDetectionService,
PeriodAccess	element/indicator label services, Ethiopian calendar + period service,
- database/ - errors/	Drift schema + per-platform connection AppException (data layer) + Failure (domain
layer)	
- metadata/ - network/	MetadataResource base + one file per DHIS2 type ApiClient (Dio), interceptors,
ConnectivityService	
- notifications/ - onboarding/ - router/ - storage/ - sync/	ReportReminderService (deadline reminders) OnboardingService, TourHelper (first-run flags) go_router config + reactive auth guard SecureStorage wrapper SyncManager, DriftSyncManager, SyncCoordinator, manual sync, SyncForegroundService,
BatteryOptimization	
- debug/ - features/ - auth/ - capture/	Dev-only screens (sync debug, in-app DB viewer) Login (online/offline) Org-unit tree → dataset → section → period;
Report	Period view; Expected-reports “To-do”; new-
- report flow - data_entry/	The entry form: bloc, table, cells, outlier +
validation	
audit_log/	UI, disease-registration list, PDF/Excel export Per-cell history sheet (local + server audit
merged)	
home/	App shell: Capture / Visualization toggle,
filters, drawer	
- onboarding/ - settings/ - visualization/	First-run carousel Server URL, profile, logout Server dashboards, local dashboards, chart
builder	
- shared/ - theme/	AppColors, AppTextStyles, AppDimensions,
AppTheme	
widgets/	Buttons, fields, loaders, FilterPanel,
SegmentedToggle,	
option grid	ServerUrlDialog, SyncSnackBar, squircle FAB,

3.3 Technology choices

Concern	Choice	Version
UI framework	Flutter / Dart	SDK >= 3.0.0 < 4.0.0; CI Flutter 3.41.4
State management	flutter_bloc (Bloc pattern, no Cubit)	^8.1.6

Navigation	go_router with a reactive login guard	^14.2.7
HTTP	Dio (background JSON transformer)	^5.7.0
Local database	Drift over SQLite (sqlite3_flutter_libs native, sqlite3.wasm on web)	drift ^2.20.0
Secure storage	flutter_secure_storage	^9.2.2
Offline credential check	crypto (SHA-256)	^3.0.5
Connectivity	connectivity_plus + an active server probe	^7.2.0
Charts	fl_chart (native rendering)	^1.2.0
Reminders	flutter_local_notifications + timezone	^17.2.3 / ^0.9.4
Onboarding	showcaseview + shared_preferences	^4.0.1 / ^2.3.2
Export	pdf + printing + excel + share_plus	—
Dev tooling	build_runner, drift_dev, flutter_lints, drift_db_viewer	—

3.4 Dependency injection

There is deliberately no DI framework (no get_it, riverpod, injectable, provider). App-wide singletons use the classic private-constructor + static instance pattern (ApiClient, SecureStorage, AppSession.instance, ConnectivityService.instance, DriftSyncManager.instance). Feature wiring is done by hand at page build time. Repository implementations take their dependencies (session, network info) as optional named parameters that default to the real singleton — which is what makes them constructible with fakes in tests without a container.

Trade-off, stated plainly: this is fully traceable (cmd-click from a widget to every dependency, no reflection or codegen) at the cost of the same wiring snippet repeating across pages. A small service locator is the natural next step if the feature count keeps growing — it is on the roadmap.

3.5 Startup sequence (main.dart)

On launch, in order: bind the Flutter engine; load onboarding / app-tour flags synchronously into memory (the router's redirect is a synchronous function and must be able to read them); lock portrait orientation; apply a Settings-stored server URL over the compiled default; start the SyncCoordinator (auto-sync, no-ops while logged out); start the ConnectivityService probe; start the chart-draft coordinator (finishes offline-saved chart drafts when connectivity returns); initialise the report-reminder service (no-op on web, self-disables if the platform refuses); then run the app. There is no splash screen and no explicit auth check here — the router's redirect combined with AppSession as a refreshListenable handles that reactively.

4 Offline-First Design

4.1 The per-user local database

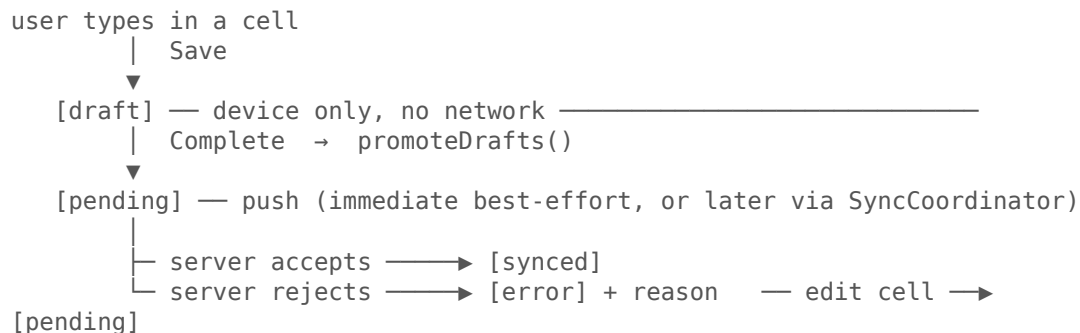
`AppDatabase.forUser(userKey)` opens a SQLite connection scoped to a sanitised username (lower-cased, non-alphanumerics collapsed to underscore). Native builds get one file per user at `<app documents>/hisp_<userKey>.sqlite`; web builds get a WASM-backed database of the same name (OPFS, falling back to IndexedDB). A single conditional export picks the platform implementation.

Why per-user files matter: a shared device with several users has fully isolated metadata caches and fully isolated pending field data. There is no cross-user query surface at all — one user's drafts cannot leak into another's session because they are not in the same database. (The one known edge case: two usernames that differ only in characters the sanitiser strips would collide.)

4.2 The write-path state machine

Every user-entered value moves through exactly these states, never skipping one. The states are stored by enum index — new states may only be appended.

State	Meaning	Sync behaviour
draft	Device-only. Written by every ordinary Save.	No sync path — automatic or manual — ever touches a draft row.
pending	Sync-eligible. Set when a form is Completed (drafts are promoted), or by conflict resolution when the local side wins.	Pushed on the next sync attempt; stays pending on transport failure.
synced	Confirmed accepted by the server.	Skipped by future pushes. Eligible for retention purge if out of window.
error	Rejected by the server; the reason is stored alongside.	Shown red in the form. Editing the cell clears the error and re-queues it as pending.



The core offline-safety guarantee: work in progress (a draft) cannot be pushed half-finished, and cannot be silently overwritten by a conflicting server value while the user is still editing it.

4.3 Conflict resolution — per cell, on form open

Whenever a form is opened online, `DataValueSync.syncForm()` pulls the server's current values for that dataset / period / org unit and resolves each cell before the form renders. This is best-effort: a pull failure just means the local data stands.

Situation	Resolution
No local value, or local already synced	Server value applied as-is, marked synced.
Local value equals server value	Marked synced, not re-transmitted.
Local value is a draft	Draft always wins. Unsubmitted device-only work is never silently replaced; it leaves the device only when the user completes the form.
Local is pending/error and differs from server — a real conflict	Device clock flagged as tampered → server wins. No server timestamp → server wins. Local lastModified is after (server lastUpdated + 2 min tolerance) → local wins (stays pending, pushed next). Otherwise → server wins.

The 2-minute tolerance is a conservative bias toward the server for near-ties — it absorbs residual clock skew without letting a genuinely stale local edit win on a technicality. After resolution, everything still pending for the form is pushed; only once nothing local remains pending anywhere is the app clock re-anchored to the server's Date header.

4.4 The tamper-resistant clock

Period-based access control (“can I still edit last month's report?”) is meaningless if the device clock can be wound back. `PeriodAccess` defends it:

- High-water mark. The latest time the app has ever observed, persisted. `effectiveNow()` never returns earlier than this mark — moving the clock back literally cannot make it go backward.
- Tamper flag. A backward jump beyond a 2-minute tolerance (to tolerate NTP corrections) sets a persistent flag. While set: entry into past periods is refused/warned; entry into the current open period continues (the data is not suspect, only backdating claims are).
- Healing. Any successful online contact anchors the clock to the server's Date header and clears the flag — gated so it will not re-anchor underneath data that is still pending.
- Every local timestamp (data-value lastModified, completion date) comes from `effectiveNow()`, not `raw DateTime.now()` — so a backdated device can never produce a timestamp earlier than something the app has already seen, which is the property the newest-wins rule depends on.

4.5 Retention

`purgeOutsideRetention(allowedPeriods)` deletes rows that are *both synced* and outside the allowed period set. Rows in pending, draft, or error state are never auto-deleted, regardless of period. A device is never allowed to silently destroy field work that has not confirmed as synced.

5 Data Capture and Quality

5.1 The capture flow

```
Home — Capture mode
  → Organisation-unit tree (lazily loaded one level at a time; ~38k units on
the
  national instance are never loaded whole)
  → Datasets assigned to the selected org unit (Routine + Disease
Registration,
  one merged list; each annotated synced / unsynced)
  → Section picker (only if the dataset has sections)
  → Period picker (gated by PeriodAccess: expiryDays / openFuturePeriods /
clock)
  → [ Disease datasets: a category-combo picker, e.g. Department × Outcome ]
  → Entry form
```

The entire path down to the form is pure local database reads — no network call happens in the capture feature. The network first appears when the form opens (a best-effort value pull) and again at sync time.

The **“Create new record” entry point** (the + FAB) is a shortcut into the same flow for starting a fresh report without walking the tree.

5.2 Ethiopian calendar and period IDs

Two files carry all date/period logic. `EthiopianCalendar` does pure calendar math via a Julian Day Number (integer day count), a fixed epoch offset, and a 4-year leap cycle; every month 1–12 has 30 days, month 13 (Pagume) has 5 or 6. It generates DHIS2-ready period IDs for every period type.

The one non-obvious part — the “Nov” period types. On a DHIS2 server running the Ethiopian calendar, “Nov” in a period-type name does not mean Gregorian November — it means Ethiopian month 11 (Hamle). These are DHIS2's Ethiopian-fiscal-year period types (fiscal year = Hamle–Sene) and their ID carries the year the fiscal year *ends* in. Verified against staging analytics.

`EthiopianCalendar.today()` is the single source of truth for “what period is current” throughout the app, independently verified against the standard (Amete Mihret) calendar. Note: the staging server's own period generation has been observed running ~2 months behind the real calendar — that is a server-side issue, not an app bug (see section 13).

5.3 Client-side value-type validation

`validateDataValue()` mirrors DHIS2's own server-side `valueType` checks (NUMBER, INTEGER, PERCENTAGE, UNIT_INTERVAL, BOOLEAN, DATE, PHONE_NUMBER, EMAIL, option-set membership, ...) and runs before a value is ever queued, so malformed data never leaves the device. It is also a second gate inside the data-entry Bloc's save path: every user-edited cell is re-validated before the save is allowed through. Unknown or free-text types are accepted as-is — the server stays the final authority.

5.4 Validation rules (offline)

`ValidationService` runs the synced DHIS2 validation rules against one form instance's local values, fully offline, on exactly the data the user sees. It contains a small expression evaluator implementing

the same subset the official DHIS2 Android SDK evaluates offline: arithmetic over data-element operands, numbers, parentheses. Anything beyond that (d2 functions, constants, indicators, [days], org-unit groups) causes the rule to be skipped rather than guessed at.

- Only rules whose period type matches the dataset and that reference at least one element of the form are considered.
- Missing-value strategies (NEVER_SKIP / SKIP_IF_ANY_VALUE_MISSING / SKIP_IF_ALL_VALUES_MISSING) are honoured; the pair operators (compulsory_pair, exclusive_pair) are evaluated on presence, not arithmetic.
- **Violations warn, they never block completion** — the same contract as the DHIS2 web and Android clients. Each violation shows the rule name, a human-readable comparison (“ANC 1st visit (12) should be ≤ ANC follow-up (10)”), the author's instruction text, and importance.
- Runs live as the user types (against Bloc state) and again at completion (against the saved local values).

5.5 Mandatory fields

Compulsory dataSetElement fields (DHIS2 dataSetElement.compulsory) and compulsory element+combo operand pairs (dataSet.compulsoryDataElementOperands) are synced and enforced. Unlike validation rules, a missing mandatory field **does block completing** the form — the check runs dataset-wide (not just the current section), again matching the DHIS2 web / Android contract.

5.6 Grey fields and controller elements

- **Grey (disabled) fields** — per-section disabled cells (DHIS2 greyedFields) are synced and rendered non-editable, exactly as the server configures them.
- **Controller data elements** — a Boolean data element tagged with the custom attribute “Controller Data Element Attribute” = true gates the visibility of the other elements in its data-element group. Example: “RMNCH – Delivery services provided” (Yes/No) shows or hides every other element in its “Delivery Services” group; answering No hides them and their entered data is cleared. Purely a metadata lookup — no network.

5.7 Outlier detection

A statistical companion to the rule check: as the user types a numeric value, it is judged against that same cell's own recent history at that org unit.

- **History snapshot.** OutlierDetectionService.fetchHistory pulls ~25 months of the dataset's values for the org unit (so a monthly dataset has two full years of the same season to compare against), reduces each cell to summary statistics, and caches the result. Offline, it falls back to the last cached snapshot. An empty result means “check disabled” — never an error.
- **The server's own /api/outlierDetection endpoint is deliberately not used** — it only scores values already stored server-side and cannot judge the value being typed right now.
- **Algorithms (user-selectable, one global setting).** Modified Z-score (median / MAD — robust to the outliers it is looking for; the default), Z-score (mean / standard deviation), and Min-Max (outside the historical range plus a guard band). Threshold is configurable; the default is 3.
- **Behaviour.** A flagged value raises a warning dialog with the typical value, the historical range, and the acceptable bounds. The user can keep the value (acknowledged once, not re-prompted)

or correct it. Values with too little history ($n < 3$) or no spread are never flagged. Settings persist in SharedPreferences.

5.8 Audit trail

The per-cell “history” sheet merges two sources into one timeline:

Source	What it is	Scope
AuditLogStore (local)	Append-only trail of edits made on this device — old value, new value, who, when. Written by <code>DataValueStore.setValue</code> and <code>CompletenessStore.setComplete</code> themselves, so every write path is covered.	This device only. Works offline.
ServerAuditService (server)	DHIS2's own audit trail via GET <code>/api/audits/dataValue</code> — authoritative history reflecting edits from every client (web, other devices). CREATE / UPDATE / DELETE per entry.	Server-wide. Requires data-value auditing enabled server-side; an empty result does not prove nothing changed.

Both are keyed to the same field shape so the two sources render identically in the sheet. A no-op is recorded for an edit that changes nothing.

5.9 Form export

An open form can be exported to PDF (via the platform print/share sheet) or to an Excel workbook (shared as a file). Useful for a paper backup or for supervisory review off-device.

6 Synchronization and Connectivity

6.1 Two connectivity concepts, on purpose

Mechanism	Answers	Used by
NetworkInfo (connectivity_plus wrapper)	"Does the OS report a network interface?"	SyncCoordinator (when to attempt a push); repositories (whether to try a best-effort server call before falling back to local).
ConnectivityService (active probe)	"Can we actually reach the server?" — probes GET {baseUrl}/system/ping every 30 s and on every OS connectivity event. Any HTTP response (including 401) counts as online; only a transport failure counts as offline.	The user-facing online/offline indicator. Uses a bare Dio client with no auth interceptor so a 401 here never ends the session.

6.2 Automatic sync — three doors, one push

SyncCoordinator.start() (called once in main) listens for three independent triggers, any of which calls the same idempotent push:

1. Offline → online transition (a connectivity change where the previous state was disconnected).
2. Login while already online (a previous offline session may have queued work with no connectivity event to catch it).
3. A 5-minute heartbeat (catches pushes stuck behind a connection that looks alive — captive portal, flaky link — that would otherwise wait forever).

pushPending() is safe to call repeatedly (sync'd rows are skipped) and no-ops while logged out.

Failures are swallowed with a debug log — a background sync must never crash the app; failed rows stay queued for the next attempt.

6.3 Manual sync

The sync button on Home / Capture runs a precise, user-facing flow: count pending work first; force-refresh connectivity (the cached flag can be 30 s stale); if offline, return "will sync automatically"; if nothing pending, return "already synced" (mentioning any drafts still on device); otherwise push, then best-effort refresh metadata, and report one of offline / nothingToSync / uploaded / partial / failed, each with a specific message.

6.4 The push and its server verdict

- Data values go out as POST /api/dataValueSets.json with importStrategy CREATE_AND_UPDATE and atomicMode NONE, capped at 500 values per request and chunked recursively — a mid-transfer drop on a flaky link only loses the unsent slice, not the whole backlog.
- DHIS2 2.38+ answers an import with rejected values with HTTP 409, but the body is still a full ImportSummary. The push treats 200 and 409 identically once it has that body: each conflict's index maps back to the exact value in the payload — an **exact per-value verdict, not a guess**.

Values not listed as rejected are marked synced; listed ones get error with the server's message. A true transport failure (no parseable summary) marks nothing — every value stays pending.

- Cleared cells are pushed as `deleted: true`, not `value: ""` (which DHIS2 rejects with E7610). A pre-push screening pass also settles type-invalid values locally before the POST.
- Completion registrations push one call each: `completed` → POST, `un-completed` → DELETE. A 409 here is treated as a permanent rejection (retrying an already-rejected registration forever cannot succeed).
- Stale dataset ↔ org-unit assignment (E7629) is recovered automatically: the picker refreshes and replaces the assignment on every online load, and a requeue pass un-sticks error rows once an admin re-assigns the dataset.

6.5 Metadata sync — full and delta

`MetadataSyncService` is the only place in the app that holds an API client for metadata; every `MetadataResource` is constructed with just a database handle, so by construction it cannot reach the network on its own.

- **Full sync** (first login into an empty database): clears all metadata / link tables, then syncs every resource in dependency order — category options → categories → combos → option combos → option sets → options → attributes → data elements → indicators → data-element groups → organisation units → data sets → sections → validation rules. The `lastMetadataSync` timestamp is written only after the full pass succeeds, so an interrupted download retries a full sync rather than getting stuck on the delta path with half-empty metadata.
- **Delta sync** (every subsequent online login): each resource fetches a cheap id + `lastUpdated` list, compares against local pairs, fetches only changed/new objects (chunked at 100 IDs per request), and deletes locally anything the server no longer has.
- **Org-unit scoping**. A facility user's metadata sync pulls only their own capture-root subtree, never the ~38,000-unit national tree. For offline storage the subtree is further bounded to the capture roots plus their direct children — a `woreda`-assigned user stores the `woreda` and its PHCUs, not every health post beneath them. Online, the full subtree stays reachable via a live fallback. Delta sync converges a device that synced before this bound existed down to the bound.
- **Failure model**. Each resource syncs in its own step; a drop mid-sequence leaves earlier resources fully synced and later ones untouched — stale but internally consistent, never half-written.

6.6 Background-kill protection (Android)

Once the app is backgrounded, Android's execution limits and — more aggressively — several OEM battery managers will kill the process mid-push. Two mechanisms mitigate this:

- **SyncForegroundService** — a native Android foreground service (`dataSync` type) started and stopped by the `SyncCoordinator` around each push attempt, so the notification only appears when there is actually work to protect. No-op on iOS (which caps background execution at a few seconds regardless).
- **BatteryOptimization** — a one-time (ever) prompt after login asking the user to exempt the app from OEM battery optimisation. The exemption still requires explicit user consent via the system dialog.

6.7 Chart draft coordinator

A local dashboard saved while offline is stored as a draft; ChartDraftCoordinator (started in main) finishes those drafts — running their analytics query and caching the result — the moment connectivity returns.

6.8 Schema for a locally-queued write table

The invariant every sync-aware class relies on: a new write-queue table carries syncState (the four-value enum), syncError (nullable text), and lastModified (sourced from PeriodAccess.effectiveNow(), never raw DateTime.now()). A transport failure leaves a row exactly where it was; only a server verdict (2xx accepted or 409 rejected) may change a row's sync state.

7 Visualization and Analytics

7.1 Three areas

The Visualization side of the Home toggle presents three flat tabs:

Tab	What it is	Connectivity
Server Dashboard	DHIS2 server dashboards, rendered natively with <code>fl_chart</code> via the same <code>/api/dashboards</code> → <code>/api/visualizations/{id}</code> → <code>/api/analytics</code> pipeline the DHIS2 web app uses. Each visualization on a dashboard is fetched and drawn independently, so one slow or broken chart never blocks the rest.	Online to load; last successful result cached for offline viewing.
Local Dashboard	Charts the user has built and saved on this device. The saved configuration lives as JSON under one key in the per-user database (no schema change); every successful query result is cached under its own key so the chart is still viewable offline. Full CRUD — create, view, edit, delete — with no server push.	Online to (re)run a chart; cached result offline.
Create New	The chart builder: pick indicators or data elements (by group, or flat from local metadata when offline), an org unit, a period, and a chart type. Saving lands the chart in Local Dashboard.	Dimension pickers prefer the live server, fall back to local metadata.

7.2 Defensive behaviour

- A visualization with no period dimension anywhere (columns / rows / filters) is refused before the analytics call — DHIS2 would answer such a query with a permanent 409 — and a specific, displayable “misconfigured visualization” message is shown instead of a generic network error.
- Relative-period flags (`last12Months`, ...) are translated to their analytics dimension IDs (`LAST_12_MONTHS`) mechanically.
- A dashboard tracks how many item types it cannot render yet (`MAP`, `TEXT`, `EVENT_CHART`, ...) so the UI can say “2 items not supported” rather than silently dropping them.
- An offline-cache banner tells the user when a chart is showing cached rather than live data.

8 Reminders, To-Do, and Onboarding

8.1 Expected reports (the “To-do” band)

`CaptureRepository.getExpectedReports()` computes, on the fly, every report the user still owes: each dataset assigned to one of their facilities (capture roots + direct children), for every period still open for entry, that is not already completed locally or on the server. It best-effort pulls the server's completion state first when online, so a report finished on the web does not linger. Results are sorted most-urgent first.

Urgency	Meaning
overdue	The period has ended but is still inside its lock window — fill it now.
dueSoon	The period is ending within a few days.
open	Open, deadline not yet close.

8.2 Deadline reminders

`ReportReminderService` schedules on-device local notifications (no server, no push) for reports due before their lock date. Rebuilt from scratch every time the expected-reports list is recomputed, so they always match reality.

- Grouped by lock date — a facility owing five reports on the same day gets one notification, not five.
- At most two per day: a heads-up three days out and a “locks today” on the day, both at 08:00.
- **Time zone is pinned to Africa/Addis_Ababa** — this is a Federal Ministry of Health app and every period boundary runs on Addis time.
- Inexact alarms only (`AndroidScheduleMode.inexactAllowWhileIdle`) — a reminder a few minutes late is fine, and this avoids the `SCHEDULE_EXACT_ALARM` permission entirely.
- No-op on web; self-disables silently if the platform refuses initialisation or permission.

8.3 Onboarding and app tour

- A first-run carousel (gated by a `SharedPreferences` flag, independent of login/logout).
- **Per-screen spotlight tours** (`showcaseview`) for Home, routine data entry, disease data entry, and Visualization — each with its own ID so it triggers once, on that screen's first visit, rather than one monolithic tour spanning navigations.
- “Take the tour again” in the Home drawer clears every screen's tour flag; Home's replays immediately, the others on next visit.

9 Security Posture

9.1 Authentication

DHIS2 uses HTTP Basic authentication (Authorization: Basic base64(username:password)). The app builds that header in exactly one place. `SessionService.login()` is the single decision tree for both login paths and returns one of: `onlineFirstSync`, `onlineReturning`, `offline`, `offlineNoCache`, `invalidCredentials`.

- **Online path:** a throwaway credentialed Dio client calls `GET /api/me.json`; 200 → valid, 401 → invalid. On success: open the per-user database; anchor the tamper-resistant clock to the server Date header (only if no local data is pending); persist the offline verifier; kick off metadata sync (full on first login, delta after) in the background.
- **Offline path:** the per-user database must already exist (otherwise `offlineNoCache` — there is no way to log in for the very first time without a connection). The verifier is recomputed and compared in constant time; a local backward-clock check runs.

9.2 The offline verifier

```
verifier = SHA-256( salt : serverUrl : username : password )
```

- salt is 16 cryptographically random bytes (`Random.secure()`), unique per user, stored alongside the hash.
- `serverUrl` is normalised and baked into the hash — **a verifier created against one DHIS2 instance cannot validate a login aimed at another.**
- Comparison is constant-time (XOR-accumulate over every byte) to resist timing attacks.
- **The password itself is never stored, in any form, online or offline.** The verifier can check a login attempt; it cannot be reversed into a password.
- Both salt and hash are held in `flutter_secure_storage`, so the verifier is encrypted at rest even though it is already a one-way hash.

9.3 Credential and session storage

Store	Holds	Backing
CredentialStore	The offline-login verifier only (salt + hash, per user).	<code>flutter_secure_storage</code>
SecureStorage	Session / profile convenience data: username, base URL, cached <code>/me</code> JSON, org units. Plus a legacy <code>auth_token</code> key that is only ever deleted (on 401 / logout), never written.	<code>flutter_secure_storage</code>
SQLite database	Cached metadata, pending field data, local audit log, local dashboard configs. The <code>UsersTable</code> caches only uid / username / displayName from <code>/api/me</code> .	Plain SQLite file (not encrypted at rest)

Platform backing for secure storage: Android → EncryptedSharedPreferences (Android Keystore-backed); iOS → Keychain (first-unlock accessibility). **Nothing sensitive lives in the SQLite database — no password, no token, no verifier.**

9.4 Logout, wipe, and 401 handling

- **Logout** closes the database connection and clears the in-memory session but keeps the database file and the offline verifier — the same user can log back in offline later without re-downloading metadata. The logout confirmation dialog explicitly tells the user that unsynced data stays on the device.
- **Wipe** deletes the database file (+ WAL/SHM sidecars) and the verifier. Guarded: it throws unless `confirmedDataLoss` is passed, after checking the count of unsynced work — callers must show that count and get explicit confirmation.
- **401 anywhere** (`AuthInterceptor`): deletes the legacy token, ends the session (not a wipe), and redirects to `/login?reason=session-expired`. The local database and verifier survive — the user can still log in offline.

9.5 Transport and platform

- HTTPS-only in release. There is no `usesCleartextTraffic` flag in the main Android manifest; the debug manifest allows cleartext explicitly, for local dev servers. iOS App Transport Security applies unmodified (no exceptions in `Info.plist`).
- Dio timeouts: 30 s each for connect / receive / send.
- **Release signing is enforced, not optional.** The Android build hard-fails the moment a `*Release` task is scheduled if `android/key.properties` is missing — this prevents an accidentally debug-signed release that can never be updated in place (a user would have to uninstall, losing unsynced data). An explicit opt-in flag exists for non-distributable local test builds.
- SQL injection is eliminated by construction — all queries go through Drift's type-safe builder; the only raw statements are fixed PRAGMAs with no user input.

9.6 Android permissions

Every permission the app requests, and why:

Permission	Purpose
INTERNET	API calls.
ACCESS_NETWORK_STATE	Connectivity detection.
FOREGROUND_SERVICE, FOREGROUND_SERVICE_DATA_SYNC	Keep an in-flight offline-data push alive after the app is backgrounded.
POST_NOTIFICATIONS	Report deadline reminders (and the sync foreground notification).
RECEIVE_BOOT_COMPLETED	Re-arm deadline reminders after a reboot. Inexact alarms only — no <code>SCHEDULE_EXACT_ALARM</code> .
REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Let the user exempt the app from OEM battery-optimisation killing. Install-time permission; the exemption itself still needs explicit user consent via the system dialog.

No storage, camera, location, contacts, or exact-alarm permissions are requested anywhere.

9.7 Logging discipline

- Release builds log warning and error only — request/response detail never reaches a field device's logcat.
- The Authorization header is explicitly redacted before any logging, and the credentialed API-client variant carries a hard rule never to log headers.
- Response bodies are logged only at debug level, truncated — and debug logging is stripped from release entirely.

9.8 Honest summary

Area	Status
Password storage	OK — never stored in any recoverable form, online or offline
Offline verification	OK — salted SHA-256, constant-time compare, server-URL-bound
Secure storage	OK — OS-level (Android Keystore / iOS Keychain)
Transport	OK — HTTPS-only in release // gap: no certificate pinning
Local database at rest	Gap — not encrypted (holds cached metadata + pending field data + local audit log; never credentials)
Android release build	OK — hard-fails without a real keystore // gaps: no R8/ProGuard minification; android:allowBackup not explicitly disabled
Logging	OK — Authorization redacted; release suppresses debug/info
SQL injection	OK — eliminated by construction (Drift only)
Session on 401	OK — ends session, redirects with a reason, preserves local data

None of the gaps expose credentials or allow remote data tampering. They are defense-in-depth items for the “device is lost or physically compromised” threat model, each with a concrete, scoped fix (section 13).

10 Database Schema

10.1 Technology

Drift over SQLite. Type-safe, compile-time-checked queries; row and companion classes are code-generated from Table subclasses — there is no hand-written SQL-to-Dart mapping anywhere. Native builds use `sqlite3_flutter_libs`; web uses `sqlite3.wasm` via a Drift worker. Journal mode is WAL; foreign keys are on.

10.2 Tables (30, current schema version 5)

Metadata tables (13) — mirror DHIS2 configuration, cleared and refreshed by a full sync

OrgUnitsTable, DataSetsTable, DataElementsTable, SectionsTable, IndicatorsTable, CategoriesTable, CategoryOptionsTable, CategoryCombosTable, CategoryOptionCombosTable, OptionSetsTable, OptionsTable, DataElementGroupsTable, ValidationRulesTable.

Link / join tables (10) — n:n relationships

DataSetElementsTable (carries sortOrder, compulsory, and an effective categoryCombo override), CompulsoryDataElementOperandsTable, DataSetOrgUnitsTable, SectionDataElementsTable, SectionIndicatorsTable, SectionGreyFieldsTable, DataElementGroupMembersTable, CategoryCategoryOptionsTable, CategoryComboCategoriesTable, CategoryOptionComboOptionsTable.

Global / control tables (4)

- UsersTable — the cached /api/me response (enables offline getCurrentUser and offline-login user reconstruction).
- AttributesTable / AttributeValuesTable — a generic custom-attribute mechanism keyed on (objectType, objectId, attributeId), so any metadata type can carry attribute values without a table-per-type explosion.
- SyncInfoTable — a key-value store for sync bookkeeping (lastMetadataSync, clock high-water mark, tamper timestamp), the outlier history cache, and local dashboard configs + result caches.

Data / write-queue tables (3) — the only tables a metadata sync never clears

- DataValuesTable — composite key (dataElementId, period, orgUnitId, categoryOptionComboId, attributeOptionComboId); value, comment, storedBy, syncState, syncError, lastModified.
- CompleteDataSetRegistrationsTable — composite key (dataSetId, period, orgUnitId, attributeOptionComboId); completed, syncState, syncError, date, lastModified.
- AuditLogTable — append-only local edit history (entityType, auditType, the entity keys, previousValue, newValue, modifiedBy, modifiedAt).

10.3 Migrations

The schema is no longer frozen. schemaVersion is 5. onUpgrade runs one ordered step per version bump; a missing step throws loudly rather than opening a field device's database against a schema it does not match (which would corrupt real data).

Version	Change
1	Frozen baseline.
2	Create AuditLogTable (local edit history).
3	Add AuditLogTable.auditType (explicit CREATE/UPDATE/DELETE); backfill existing rows from previousValue/newValue nullness.
4	Add DataSetElementsTable.compulsory — mandatory-field support.
5	Create CompulsoryDataElementOperandsTable — the element+combo-pair sibling of dataSetElement.compulsory.

Steps 3 and 4 guard against a device that crashed mid-migration by checking column existence first, so a retry does not fail forever on “duplicate column”. Adding a table means: define it, add it to the @DriftDatabase list, add it to the metadata-clear list if it is metadata, bump schemaVersion with a step, and regenerate.

10.4 Testing against the database

Tests use an in-memory database (AppDatabase.forTesting(NativeDatabase.memory())) — no file I/O, no platform plugins — which is what lets the full test suite run in plain CI with no emulator.

11 Build, Release, and CI

11.1 Continuous integration

`.github/workflows/ci.yml` runs on every push to `main` / `integration*` and every pull request: `flutter pub get` → `flutter analyze` → `flutter test`. Pinned to Flutter 3.41.4. A red CI blocks merge. There is no build/deploy job in the workflow — analyze + test only.

11.2 Android release build

Configured in `android/app/build.gradle.kts`. Release builds require a real signing keystore and the build hard-fails without one.

4. Generate a keystore once (`keytool -genkey ...`). Guard the file and password — an Android app can only ever be updated by APKs signed with the same key.
5. Create `android/key.properties` (gitignored) with `storeFile` / `storePassword` / `keyPassword` / `keyAlias`.
6. `flutter build apk --release`.

A non-distributable debug-signed test APK can be built with an explicit `opt-in` (`ALLOW_DEBUG_SIGNING=true`), which logs a loud warning. Not yet configured: R8/ProGuard minification; `android:allowBackup="false"`.

11.3 Web build

`flutter build web`. The WASM database assets (`sqlite3.wasm`, `drift_worker.js`) are served from `web/` and are version-pinned to the `drift` / `sqlite3` package versions — re-download them together when bumping either. For local development against staging use `./run_web.sh`, which pins the dev server to port 3001 (the only port besides 3000 that staging's CORS policy allows).

11.4 Code generation

The Drift database uses generated code (`app_database.g.dart`, ~16,900 lines, committed). Regenerate only after editing a table definition: `dart run build_runner build --delete-conflicting-outputs`.

12 Testing

24 test files, ~5,200 lines, mirroring lib/ under test/. Coverage is deliberately weighted toward the offline-critical core — the code where a bug would silently corrupt or lose field data, and the code hardest to verify by clicking through the UI.

Well covered

- Sync engine and conflict resolution (data_value_store, data_value_sync, data_value_push, completeness + completeness pull).
- Data quality — value-type validator, validation service (rule evaluation), outlier detection, controller-element service, element/indicator label services.
- Ethiopian calendar and period service.
- Metadata resources (data set, organisation unit, section) and Drift round-tripping.
- Capture repository (including expected-reports computation), local visualization repository, disease-entry list, PDF export, date-filter resolution.

Thin

- Bloc-level tests (AuthBloc, DataEntryBloc have no dedicated files) and full widget / integration tests beyond smoke-level rendering.

Conventions

- In-memory database for anything touching AppDatabase.
- Network-touching classes take an ApiClient by constructor injection; tests supply a canned/fake HTTP adapter. No mockito / mocktail — the project favours real fakes and in-memory implementations.

13 Roadmap and Known Limitations

Stated transparently. None of these involve credential exposure or remote data tampering.

13.1 Security hardening (before large-scale rollout)

7. Encrypt the local database at rest (SQLCipher-backed SQLite). It holds cached metadata, pending field data, and the local audit log — never credentials — but a lost device should still not expose facility health data in a plain-readable file.
8. Enable R8/ProGuard minification for Android release builds.
9. Certificate pinning for the production DHIS2 host.
10. Set `android:allowBackup="false"` — health data should not ride Android's default auto-backup.

13.2 Engineering investment

11. Bloc and widget test coverage (the offline-critical core is well tested; the Blocs and most widgets are not).
12. A lightweight service locator (`get_it`) to remove the repeated manual-DI wiring across pages.
13. Crash and performance monitoring (there is currently no automated crash reporting).

13.3 Product roadmap (depends on Ministry / HISP priorities)

- Biometric app-lock for at-rest device access.
- Role-based dashboards; a fuller in-app audit view.
- A move from HTTP Basic to OAuth2 / PAT token auth once the target DHIS2 deployment supports it.
- Multi-language support (Amharic, Afaan Oromo, Tigrinya) — the app is currently English-only.
- Dark mode.

13.4 Known, accepted trade-offs

- Manual dependency injection instead of a framework — simple and traceable, at the cost of repeated wiring code.
- The data-entry UI does not support non-default attribute category combos for Routine datasets — every value is stored under the dataset instance's default combo (Disease Registration datasets do use a real combo, picked before the form opens). A scope decision, not a bug.
- Server-side validation rule features beyond basic arithmetic (d2 functions, constants, indicators, org-unit groups) are skipped by the offline evaluator rather than approximated.
- Indicator groups are not synced for offline use (only indicators themselves), so the offline chart builder falls back to a flat indicator list.
- `OFFLINE_INTEGRATION.md` at the repo root is a pre-implementation design note, superseded by the current implementation — kept for historical context only.

13.5 Staging-environment issues (not app bugs)

- **The staging server's own “current period” has been observed running ~2 months behind the real Ethiopian calendar** (investigated 2026-07-21). Data pushed “for today” lands correctly but the staging web UI will not display that period for weeks. The app's calendar conversion is the correct one — verified two independent ways; the server's period generation is what is behind. Check via the API directly before assuming a sync bug.

- The staging “Routine Data Entry” web app's section tabs do not switch past the first — a front-end bug in that app, unrelated to this codebase.

Appendix A Glossary

Term	Meaning
DHIS2	District Health Information System 2 — the platform this app feeds.
Aggregate data	Counts and totals for a period (as opposed to individual-level tracker data). This app is aggregate-only.
Data element	One thing being counted (e.g. “ANC 1st visit”).
Category option combo (COC)	A disaggregation of a data element (e.g. “<1 year, Male”). Forms have data elements as rows and COCs as columns.
Attribute option combo (AOC)	A whole-form disaggregation (e.g. a funding stream, or Disease Registration's Department × Outcome).
Data set	A form: a set of data elements captured together for a period and org unit.
Organisation unit (org unit)	A node in the health facility hierarchy (country → region → zone → woreda → facility).
Period	The reporting window (a month, quarter, year, ...), identified by a DHIS2 period ID.
expiryDays	Days after a period ends during which the server still accepts data for it. 0 = never locks.
openFuturePeriods	How many not-yet-started periods the server will accept data for.
Completion registration	A record that a form was signed off as complete for a period / org unit.
Validation rule	A server-defined cross-check between data elements (e.g. $A \leq B$). Warns, does not block.
Grey field	A cell the server configures as disabled for a section.
Drift	The Dart persistence library wrapping SQLite used here.
Bloc	The state-management pattern (events in, states out) used throughout.

Appendix B Key File Reference

A newcomer's map to where each concern actually lives.

Concern	File(s)
App entry / startup	lib/main.dart
Login decision tree	lib/core/auth/session_service.dart
Offline verifier	lib/core/auth/credential_store.dart

Session + router guard	lib/core/auth/app_session.dart, lib/core/router/app_router.dart
HTTP client + interceptors	lib/core/network/api_client.dart, lib/core/network/interceptors/
Connectivity	lib/core/network/network_info.dart, lib/core/network/connectivity_service.dart
Database schema + migrations	lib/core/database/app_database.dart
Metadata sync	lib/core/metadata/metadata_sync_service.dart, lib/core/metadata/metadata_resource.dart
Sync engine	lib/core/sync/ (drift_sync_manager, sync_coordinator, manual_sync, sync_foreground_service, battery_optimization)
Write path + conflict resolution	lib/core/data/data_value_store.dart, data_value_sync.dart, data_value_push.dart, completeness.dart
Tamper-resistant clock	lib/core/data/period_access.dart
Ethiopian calendar	lib/core/data/ethiopian_calendar.dart, ethiopian_period_service.dart
Value-type validation	lib/core/data/value_type_validator.dart
Validation rules (offline)	lib/core/data/validation_service.dart
Outlier detection	lib/core/data/outlier_detection_service.dart, lib/features/data_entry/domain/entities/outlier_stats.d art
Controller elements	lib/core/data/controller_element_service.dart
Audit trail	lib/core/data/audit_log_store.dart, server_audit_service.dart, lib/features/audit_log/
Deadline reminders	lib/core/notifications/report_reminder_service.dart
Onboarding / tour	lib/core/onboarding/, lib/features/onboarding/
Capture flow	lib/features/capture/
Entry form	lib/features/data_entry/
Visualization	lib/features/visualization/
Developer docs	docs/ (16+ chapters, file-cited)

End of document. Prepared from the repository at the current development head, 10 September 2026. For the most current behaviour of any specific claim, verify against the file cited.